

Ajloun National University
College of Information Technology

Database Lab

Department of Computer Science

Introduction to Database Lab

Prepared by: Eng. Norhan Zregat

Data Definition Language (DDL) : as a standard for commands through which data structures are defined. It is a computer language that is used for creating and modifying structure of the database objects, such as schemas, tables, views, indexes…etc

DDL for tables:

- The SQL create table statements.
- The SQL alter table statements.
- The SQL drop table statement.

1) The create table statement

```
CREATE TABLE table_name
(
column_name1
data_type,
column_name2
data_type,
column_name3
data_type,
```

SQL Data Types:

1. **VARCHAR (size),** VARCHAR2(size) : hold string with size characters
2. **CHAR (size)**: hold fixed length string
3. **DATE**: Stores a date like June 30, 1991
4. **NUMBER** (size): Store an integer value
5. **NUMBER** (precision, scale): store a double value
6. **INTEGER**: Store whole numbers
7. **FLOAT**: Store floating point numbers
8. **TIMESTAMP**: Stores date and time

SQL Constraints are rules applied to table columns in a database to ensure the accuracy, reliability, and integrity of the data stored in the tabl

No.	Constraint Type	Description
1	NOT NULL	Ensures that a column cannot contain NULL values.
2	UNIQUE	Ensures that all values in a column are different.
3	PRIMARY KEY	Uniquely identifies each record in a table. It must be unique and cannot contain NULL values.
4	FOREIGN KEY	Links one table with another table and maintains referential integrity between them.
5	CHECK	Ensures that the values in a column satisfy a specific condition.
6	DEFAULT	Provides a default value for a column when no value is specified.

Example :

SQL Example with All Constraints

```
CREATE TABLE Students (
    ID NUMBER,
    Name VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Age NUMBER,
    Country VARCHAR(50) DEFAULT 'Jordan',
    DeptID NUMBER,

    CONSTRAINT pk_student PRIMARY KEY (ID),
    CONSTRAINT chk_age CHECK (Age >= 18),
    CONSTRAINT fk_department FOREIGN KEY (DeptID)
    REFERENCES Department(ID)
);
```

Exercise:

Q1: Create a table named **Employees** with the following requirements:

1. EmpID → Primary Key
2. FirstName → Cannot be NULL
3. LastName → Cannot be NULL
4. Email → Must be unique
5. Age → Must be 18 or older
6. DepartmentID → References Departments(DeptID)
7. Country → Default value should be 'USA'

Instructions:

- Write the **CREATE TABLE** statement with all the constraints included.
- Use proper naming for each constraint.

The SQL ALTER TABLE Statement

1) ADD COLUMN IN TABLE

To add a column in a table, the SQL ALTER TABLE syntax is:

```
ALTER TABLE table_name   ADD column_name column-Datatype;
```

2) ADD MULTIPLE COLUMNS IN TABLE

To add multiple columns to an existing table, the SQL ALTER TABLE syntax is:

```
ALTER TABLE table_name
 ADD (column_1 column-definition,
    column_2 column-definition,
    …
    column_n column_definition);
```

3) MODIFY COLUMN IN TABLE

To modify a columns' data type in an existing table, the SQL ALTER TABLE syntax is:

```
ALTER TABLE table_name   MODIFY column_name column_type;
```

4) MODIFY MULTIPLE COLUMNS IN TABLE

To modify multiple columns(Data Type) in an existing table, the SQL ALTER TABLE syntax is:

```
ALTER TABLE table_name
 MODIFY (column_1 column_type,
      column_2 column_type,
       …
       column_n column_type);
```

5) ADD CONSTRAINT TO TABLE

General Structure of (ALTER TABLE) to Add a Constraint

The general syntax to add a constraint using ALTER TABLE is:

```
ALTER TABLE table_name
ADD CONSTRAINT constraint_name constraint_type (column_name);
```

Where:

- table_name → The table you want to modify
- constraint_name → Name you give to the constraint
- constraint_type → Type of constraint (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK)
- column_name → The column(s) on which the constraint is applied

Example: Adding a Constraint

```
ALTER TABLE Students
ADD CONSTRAINT pk_student PRIMARY KEY (ID);
```

Explanation:

- Students → Table being modified
- pk_student → Name of the constraint
- PRIMARY KEY → Type of constraint
- (ID) → Column to which the constraint applies

Exercise

Q2:**You have a table called** Students **with the following columns:**

- ID (NUMBER)
- Name (VARCHAR2(50))
- Age (NUMBER)

Perform the following tasks using **ALTER TABLE statements**:

1. Add a new column Email with data type VARCHAR2(100).
2. Modify the data type of Name to VARCHAR2(100).
3. Add a **PRIMARY KEY** constraint to ID.
4. Add a **CHECK** constraint to ensure Age >= 18.

Instructions:

- Write separate **ALTER TABLE statements** for each task.
- Use simple and clear constraint names.

Database Lab – Lab (3)

1) DROP COLUMN in Table

To remove a column from an existing table, the syntax is:

```
ALTER TABLE table_name
DROP COLUMN column_name;
```

2) RENAME COLUMN in Table

To rename a column in an existing table, the syntax is:

```
ALTER TABLE table_name
RENAME COLUMN old_column_name TO new_column_name;
```

3) RENAME TABLE

To rename a table, the syntax is:

```
ALTER TABLE old_table_name
RENAME TO new_table_name;
```

4) DROP TABLE

```
DROP TABLE table_name;
```

To delete an entire table from the database, the syntax is:

5) TRUNCATE TABLE

To remove all data from a table but keep the table structure, the syntax is:

```
TRUNCATE TABLE table_name;
```

Exercise 3:

Consider the following table and answer the following Questions:

```
CREATE TABLE customer
( customer_id  number(10),
  customer_name varchar(50),
  address varchar(50),
  city varchar(50) );
```

A) Add two columns, phone as a varchar(12) and email as a varchar (20) to the customer table.

B) Rename the customer_id column to id.

C) Rename the customer table to custom.

D) Drop the city column.

E) Change the data type of phone column to number(10).

Homework 1:

Based on the DDL SQL commands, Answer the following Questions:

Q1. Write a query to create a table employee with empno, ename, job, and salary.

CREATE TABLE employee (EMPNO NUMBER (4), ENAME VARCHAR2 (10), job

VARCHAR2 (10), SALARY NUMBER (8,2));

Q2. Write a query to create a new table called emp1 from an existing table emp with all the fields.

Q3. Write a Query to change the size of column EMPNO in employee from NUMBER(4) TO NUMBER(6).

Q4. Write a query to add a new column called QUALIFICATION to employee table of type string 20.

Q5. Write a query to drop a column job from an existing table employee.

Q6. Write a query to drop an existing table employee.

Database Lab – Lab (4)

Data Manipulation Language (DML) Commands

SQL statements that can be used to manipulate data in tables and views.

1) INSERT

The **INSERT INTO** statement is used to **add new rows of data** into a table.

Syntax:

a) Insert specifying columns

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

b) Insert entire row (all columns)

```
INSERT INTO table_name
VALUES (value1, value2, value3, ...);
```

Hints / Notes:

- If a column has a **DEFAULT value** or allows **NULL**, it can be skipped in the insert statement.
- The **number of values** must match the number of columns specified (or total columns if skipping names).
- Strings and dates should be enclosed in **single quotes** ' '.
- Columns with **AUTO_INCREMENT / SEQUENCE** values can be omitted if the database automatically generates them.

Examples:

1) Insert specifying columns

```
INSERT INTO Students (ID, Name, Age, Country)
VALUES (1, 'Razan', 20, 'Jordan');
```

2) Insert entire row

```
INSERT INTO Students
VALUES (2, 'Kamel', 22, 'Palestine');
```

```
INSERT INTO Students (ID, Name, Age)
VALUES (3, 'Lina', 19);
```

3) Insert skipping column with default

Here, Country will take the **default value** if defined in the table.

4) Insert multiple rows at once

```
INSERT INTO Students (ID, Name, Age, Country)
VALUES
(4, 'Omar', 21, 'Egypt'),
(5, 'Sara', 23, 'Lebanon'),
(6, 'Ali', 20, 'Jordan');
```

Sample Question – INSERT Statement (DML)

Q4: You have a table named **Students** with the following columns:

- ID (NUMBER)
- Name (VARCHAR2(50))
- Age (NUMBER)
- Country (VARCHAR2(50))

Perform the following tasks using **INSERT statements**:

1. Insert a single student with:
   - ID = 1
   - Name = 'Razan'
   - Age = 20
   - Country = 'Jordan'
2. Insert another student with:
   - ID = 2
   - Name = 'Kamel'
   - Age = 22

   (Use the default value for Country if defined.)

3. Insert multiple students at once with the following data:

ID	Name	Age	Country
3	Lina	19	Jordan
4	Omar	21	Egypt
5	Sara	23	Lebanon

Instructions:

- Write separate INSERT INTO statements for tasks 1 and 2.
- Write a **single INSERT INTO statement** for task 3.

Database Lab – Lab (5)

2) UPDATE – Data Manipulation Language (DML)

The **UPDATE** statement is used to **modify existing data** in a table. It allows you to change the values of one or more columns for specific rows or for all rows in the table.

1) Update specific rows

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE some_column = some_value;
```

2) Update all rows in a column

```
UPDATE table_name
SET column1 = value;
```

Hint: If you omit the WHERE clause, the same value will be applied to **all records** in the column.

Rules / Notes

1. Always use a **WHERE clause** to avoid updating all rows unintentionally.
2. Multiple columns can be updated at once by separating them with commas.
3. Values must match the column's **data type**.

Examples

1) Update a specific student's age

```
UPDATE Students
SET Age = 21
WHERE ID = 1;
```

2) Update multiple columns for a specific student

```
UPDATE Students
SET Name = 'Razan Obeidat', Country = 'Jordan'
WHERE ID = 1;
```

3) Update all students' country to 'USA'

```
UPDATE Students
SET Country = 'USA';
```

This will change the Country for **all rows** in the table.

Sample Question – UPDATE Statement (DML)

Q5: You are working with a table named **Students** with the following columns:

- ID (NUMBER)
- Name (VARCHAR2(50))
- Age (NUMBER)
- Country (VARCHAR2(50))

Tasks:

1. Create the Students table.
2. Insert at least three sample students into the table.
3. Update the **Age** of the student with ID = 1 to 21.
4. Update the **Name** and **Country** of the student with ID = 1 to 'Razan Obeidat' and 'Jordan'.
5. Update the **Country** of **all students** to 'USA'.

Instructions:

- Write SQL statements for each task in order.
- Use the WHERE clause where appropriate to update specific rows.
- Make sure the table has initial data before performing the updates.

3) DELETE – Data Manipulation Language (DML)

The **DELETE** statement is used to remove rows from a table.
It allows you to delete specific records using a condition or delete all records from the table.

1) Delete specific rows

```
DELETE FROM table_name
WHERE some_column = some_value;
```

2) Delete all rows from a table

```
DELETE FROM table_name;
```

Hint: If you omit the **WHERE** clause, all rows in the table will be deleted, but the table structure will remain.

Rules / Notes

1. Always use a **WHERE** clause when you want to delete specific rows.
2. If **WHERE** is not used, all records in the table will be deleted.
3. The **DELETE** statement removes data but does not remove the table structure.
4. Deleted data cannot be recovered unless a backup exists.

Examples

1) Delete a specific student

```
DELETE FROM Students
WHERE ID = 2;
```

This will delete the student whose **ID = 2**.

2) Delete students from a specific country

```
DELETE FROM Students
WHERE Country = 'USA';
```

This will remove all students whose country is **USA**.

3) Delete all students from the table

```
DELETE FROM Students;
```

This will remove **all rows** in the table, but the **Students** table will still exist.

Sample Question – DELETE Statement (DML)

Q6: You are working with a table named Students with the following columns:

- ID (NUMBER)
- Name (VARCHAR2(50))
- Age (NUMBER)
- Country (VARCHAR2(50))

Tasks:

1. Create the **Students** table.
2. Insert at least **three sample students** into the table.
3. Delete the student whose **ID = 2**.
4. Delete all students whose **Country = 'USA'**.
5. Delete all records from the **Students** table.

Instructions:

- Write **SQL statements** for each task in order.
- Use the **WHERE clause** where necessary.
- Ensure the table contains **data before performing DELETE operations**.

Database Lab – Lab (6)

DCL – Data Control Language

Data Control Language (DCL) is used to manage **access permissions** in a database.
It controls **who can use the data** and **what operations they are allowed to perform**.

DCL does not deal with creating or modifying data directly. Instead, it focuses on **security and user authorization**.

1) GRANT

The **GRANT** statement is used to **give permissions** to a user.
These permissions define what actions the user can perform on a table.

Syntax:

```
GRANT privilege
ON table_name
TO user_name;
```

Explanation

- **privilege**: the type of access (SELECT, INSERT, UPDATE, DELETE)
- **table_name**: the table on which permission is given
- **user_name**: the user receiving the permission

Examples:

Grant SELECT permission

```
GRANT SELECT
ON Students
TO user1;
```

This allows the user to read data from the table.

Grant multiple permissions

```
GRANT SELECT, INSERT, UPDATE
ON Students
TO user1;
```

This allows the user to read, insert, and update data.

Grant all permissions

```
GRANT ALL
ON Students
TO user1;
```

This gives full access to the table.

2) REVOKE

The **REVOKE** statement is used to **remove permissions** from a user.
It cancels privileges that were previously granted.

Syntax

```
REVOKE privilege
ON table_name
FROM user_name;
```

Explanation

- **privilege**: the permission to remove
- **table_name**: the table affected
- **user_name**: the user losing the permission

Examples:

1. Revoke SELECT permission

```
REVOKE SELECT
ON Students
FROM user1;
```

This prevents the user from reading data.

2. Revoke multiple permissions

```
REVOKE INSERT, UPDATE
ON Students
FROM user1;
```

This removes the ability to insert and update data.

3. Revoke all permissions

```
REVOKE ALL
ON Students
FROM user1;
```

This removes all access to the table.

Rules / Notes

1. DCL is used for controlling access, not for modifying data.
2. Only authorized users (such as administrators) can use GRANT and REVOKE.
3. Multiple permissions can be granted or revoked in one statement.
4. Permissions should be assigned carefully to protect data security.

Sample Question – DCL

You are working with a table named Students.

Tasks:

1. Grant SELECT permission on the Students table to user1.
2. Grant INSERT and UPDATE permissions to user1.
3. Revoke UPDATE permission from user1.
4. Revoke all permissions from user1.

Assignment

Write SQL statements to complete the following:

1. Grant SELECT and DELETE permissions on the Students table to a user named user2.
2. Grant ALL permissions to user2.
3. Revoke DELETE permission from user2.
4. Revoke all permissions from user2.

Database Lab – Lab (7)

4) *SELECT – Data Query Language (DQL)*

The **SELECT** statement is used to retrieve data from a database.
It allows you to select specific columns or all columns from a table.

1) Select specific columns

```
SELECT column_name1, column_name2
FROM table_name;
```

2) Select all columns from a table

```
SELECT *
FROM table_name;
```

Hint: The **(*)** symbol means selecting **all columns** from the table.

Rules / Notes

1. The **SELECT** statement is used to retrieve data from a table.
2. You can select **specific columns** or **all columns** using (*).
3. The **WHERE** clause can be used to filter specific rows.
4. Column names must exist in the table.

Examples:

1) Select all data from the Students table

```
SELECT *
FROM Students;
```

This will display **all columns and rows** in the table.

2) Select specific columns

```
SELECT Name, Age
FROM Students;
```

This will display only the **Name** and **Age** columns.

3) Select students from a specific country

```
SELECT *
FROM Students
WHERE Country = 'Jordan';
```

This will display students whose **Country is Jordan**.

Sample Question – SELECT Statement (DQL)

Q7: You are working with a table named **Students** with the following columns:

- ID (NUMBER)
- Name (VARCHAR2(50))
- Age (NUMBER)
- Country (VARCHAR2(50))

Tasks:

1. Create the **Students** table.
2. Insert at least **three sample students** into the table.
3. Display **all columns** from the Students table.
4. Display only the **Name and Age** columns.
5. Display students whose **Country = 'Jordan'**.

Instructions:

- Write **SQL statements** for each task in order.
- Use **SELECT** to retrieve data.
- Ensure the table contains **data before performing SELECT queries**.

WHERE Clause

The **WHERE clause** is used to filter records in a query.
It extracts only the rows that meet a specified condition.

Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE column_name operator value;
```

Basic Operators

Operator	Description
=	Equal
<>	Not equal
>	Greater than
<	Less than
3>=	Greater than or equal
<=	Less than or equal

Examples

1) Select students whose Country is Jordan

```
SELECT *
FROM Students
WHERE Country = 'Jordan';
```

This will display students whose **Country equals Jordan**.

2) Select students whose Age is greater than 20

```
SELECT *
FROM Students
WHERE Age > 20;
```

This will display students **older than 20**.

3) Select students whose Age is less than or equal to 21

```
SELECT *
FROM Students
WHERE Age <= 21;
```

This will display students whose **Age is 21 or younger**.

4) Select students whose Country is not USA

```
SELECT *
FROM Students
WHERE Country <> 'USA';
```

This will display students **not from USA**.

Rules / Notes

1. The **WHERE clause** is used to filter specific records.
2. It is commonly used with **SELECT, UPDATE, and DELETE**.
3. The condition must use a **valid operator**.
4. Text values must be written inside **single quotes (' ')**.

Sample Question – WHERE Clause

Q8: You are working with a table named **Students** with the following columns:

- ID (NUMBER)
- Name (VARCHAR2(50))
- Age (NUMBER)
- Country (VARCHAR2(50))

Tasks

1. Create the **Students** table.
2. Insert at least **three sample students** into the table.
3. Display students whose **Country = 'Jordan'**.
4. Display students whose **Age > 20**.
5. Display students whose **Country <> 'USA'**.

Instructions

- Write **SQL statements** for each task in order.
- Use the **WHERE clause** to filter records.
- Make sure the table contains **data before running the queries**.

Homework:

Create a folder with your ID number on the desktop, and do the following:

1. Create a table courses with the following properties:
   a. Id_number, with 8 numeric digits.
   b. St_Name, with 15 character string.
   c. Course_name with 15 character string.
   d. Department, with 3 character string.
   e. Bdate, date type.
   f. 33Mark, with 3 numeric digits.

3

2. Insert the following data to courses table:

Id_number	St_Name	Course_name	Department	Bdatee	Mark
11	ALi	DB	CSA	14-2-2013	70
22	Ahmad	DB	CIS	15-3-2012	40
33	Muna	VP	SWE	11-4-2012	90

3. Modify the courses table by adding the following columns:
a. Gpa, with 5 numeric digits size, 2 decimal points.
b. Flag, with 1 digit number.

4. Modify the courses table by delete gpa column.

5. Create table new_courses from the table courses.

6. Delete the table new_courses from your Database.

7. Display all data contains in courses table.

8. Delete all data of id_number =11.

9. Write and run the suitable SQL command to display only st_name and mark for all students on courses table.

10. Put the value 1 in flag column for all students in courses table .

11. Write and run the suitable SQL command to display only id_number, st_name, bdate, and mark for all students on CIS department.

Database lab (8)

Operators in select statement

Alias Names : is the process of renaming a column(s) temporarily by giving another name .

Syntax:

```
SELECT column_name as  alias_name   FROM     table_name;
OR
SELECT column_name  " alias_name" FROM     table_name
```

Concatenation Operator || : Concatenates character strings.
Example:

```
SELECT first_name ||' ' || last_name  as  "Full name"  FROM     employees;
```

Arithmetic operators are used to perform mathematical functions in SQL—the same as in most other languages. There are four conventional operators for mathematical functions:

- + (addition)
- - (subtraction)
- * (multiplication)
- /

(division)

```
SELECT salary, salary*12 " annual salary"  FROM     employees;
```

DISTINCT keyword :is used to return only distinct (different) values.

Syntax:

```
SELECT  DISTINCT column1  FROM table_name ;
```

Exercise 1: use(employees table in hr)

1) Write a query to display employee id, first name, hire Date for all employees. Name the column headings : EMP#, EMPLOYEE and StartDate .

2) Write a query to display first name, job id for all employees. Display the first name concatenated with the job id(separated by comma and space)and name the column Employee and Title.

3) Write a query to display first name, last name and a column called ” Total income” for all employees. Total income is calculated by multiplying monthly salary with 12, plus onetime bonus $100.

4) Write a query to display job id without duplicated rows.

SELECT and WHERE Clause: The WHERE clause is used to extract only those records that fulfill a specified criterion.
Syntax:
SELECT column_name(s) FROM table_name ***WHERE column_name operator value***;

Operator	Description
=	Equal
<>	Not equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
BETWEEN	Between an inclusive range
LIKE	Search for a pattern
IN	To specify multiple possible values for a column

BETWEEN operator: is used to select a range. The syntax for the BETWEEN operator is as follows:

```
SELECT column1, column2,…. FROM table_name WHERE column_name between value1 and value2;
```

IN operator: is used to select one or more of the values in parenthesis.
The syntax for using the IN keyword is as follows:

```
SELECT column1, column2, …. FROM table_name WHERE column_name IN (value1 ,value2,….);
```

Like operator: is used to compare a value to similar values using wildcard operators. There are two wildcards used:

1) The percent sign (%)represents zero, one, or multiple characters
2) The underscore (_) represents a single number or character.

Syntax:

```
SELECT column1, column2, …FROM table_name
WHERE column_name LIKE ‘ pattern’;
```

Exercise 2: use(employees table in hr)

1) Write a query to display first name, last name and hire date for all employees who were hired before 07-03-2006.

2) Write a query to display employee id, first name, hire date for all employees who joined Between January, 2002 and December, 2002.

3) Write a query to display the first name, last name and salary for all employees whose salary is between $2,500 and $3,500.

4) Write a query to display employee id, first name, last name, salary and manager’s employee number for all employees whose manager’s employee number is 100, 101 or 201.

5) Write a query to display employee id, first name, last name, and Department id for all employees whose last name is included in the following list of name [Hartstein, Vargas, Hunold].

6) Write a query to display last name, job id for all employees whose Job id is not in [IT_PROG, ST_CLERK, SA_REP].

7) Write a query to display employee id, first name, last name for all employees whose last name have the letter "o" as the second character .

8) Write a query to display employee id, first name, last name for all employees whose first name begins with the letter "O" .

The SQL AND, OR and NOT Operators

The WHERE clause can be combined with AND, OR, and NOT operators that help to include two or more conditions.

The **WHERE clause** can be combined with **AND, OR, and NOT operators** to apply **multiple conditions** in a query.

These operators help filter data more precisely.

1) AND Operator

The **AND** operator is used to combine two or more conditions.
It returns records **only if all conditions are true**.

Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE condition1 AND condition2;
```

Explanation

Both conditions must be true for a row to be included in the result.

Example:

```
SELECT *
FROM Students
WHERE Country = 'Jordan' AND Age > 20;
```

This returns students who:

- are from Jordan
- and older than 20

2) OR Operator

The **OR** operator is used to combine multiple conditions.
It returns records **if at least one condition is true**.

Syntax

```
SELECT column_name(s)
FROM table_name
WHERE condition1 OR condition2;
```

Explanation

Only one condition needs to be true.

Example

```
SELECT *
FROM Students
WHERE Country = 'Jordan' OR Age > 20;
```

This returns students who:

- are from Jordan
- or older than 20

3) NOT Operator

The **NOT** operator is used to **reverse a condition**.
It returns records that **do not meet the condition**.

Syntax

```
SELECT column_name(s)
FROM table_name
WHERE NOT condition;
```

Example

```
SELECT *
FROM Students
WHERE NOT Country = 'USA';
```

This returns students who are **not from USA**.

Combined Example

You can combine operators together:

```
SELECT *
FROM Students
WHERE Country = 'Jordan' AND NOT Age < 18;
```

This returns students who:

- are from Jordan
- and are not younger than 18

Rules / Notes

1. AND requires **all conditions to be true**.
2. OR requires **at least one condition to be true**.
3. NOT reverses the condition.
4. These operators are used inside the WHERE clause.
5. Conditions can be combined for more precise filtering.

Sample Question

You are working with a table named Students.

Tasks:

1. Display students whose Country = 'Jordan' AND Age > 20.
2. Display students whose Country = 'Jordan' OR Age > 20.
3. Display students whose Country is NOT 'USA'.

4. Display students whose Age is greater than 18 AND NOT from 'USA'.

Assignment

Write SQL queries for the following:

1. Select students whose Age is greater than 18 AND less than 25.
2. Select students whose Country is 'Jordan' OR 'USA'.
3. Select students whose Age is NOT equal to 20.
4. Select students whose Country is 'Jordan' AND Age is less than or equal to 22.

Exercise 3: use(employees table in hr)

1) Write a query to display employee id, first name, last name for all employees who have a job title contains the string 'MAN' and earn 1000$ or more.

2) Write a query to display employee id, first name, job id for all employees who have a job title contains the string 'MAN' or earn 1000$ or more.

3) Write a query to display employee id, first name, job if employee job is (AD_PRES) or (SA_REP), and if employee salary is more than 15000$.

The SQL ORDER BY Keyword: is used to sort the result-set in ascending or descending order. The ORDER BY keyword sorts the records in ascending order by default. To sort the records in descending order, use the DESC keyword. ORDER BY is written at the end of the command statement followed by column name.

Exercise 4: use(employees table in hr)

1) Write a query to display employee id, first name and department id. The result should be sorted by department id in descending order.

2) Write a query to display employee id, first name and department id. Order the query in ascending alphabetical order by first name.

Exercise 5: use(employees table in hr)

1) Write a query in SQL to display the details of the employees who have salary within the range 7000 to 12000 and works in department number is 50.

2) Write a query in SQL to display the employee ID, first name, job id, and department number for those employees who is working except the departments 50,30 and 80.

3) Write a query in SQL to display the first and last name, and department number for those employees who holds a letter ‘s’ as a 3rd character in their first name.

4) Write a query in SQL to display the full name (first name and last name), hire date, commission percentage, email and telephone separated by '-', and salary for those employees who earn the salary above 11000 or the seventh digit in their phone number equals 3 and make the result set in a descending order based on first name.

5) Write a query in SQL to display the full name (first and last), job id and date of hire for those employees who were hired during March 5th, 2004 and July 5th, 2005.

6) Write a query in SQL to display the full name (first and last) name, and salary, for all employees whose salary is out of the range 7000 and 15000 and make the result set in ascending order by the first name.

7) Write a query in SQL to display all the information for all employees who have the letters D, S, or N in their first name and also arrange the result in descending order by salary.

Database Lab (9+10)

SQL FUNCTION:

SQL NULL is the term used to represent a missing value. A NULL value in a table is a value in a field that appears to be blank. We use the **IS NULL** and **IS NOT NULL** operators to check if a field is blank or not.

1- **NVL(exp1,exp2):** The NVL function permits you to replace all null values with a default value. It requires two parameter values first value is column name and second is any default value. If the value in the first parameter is NULL, then the function returns the value in the second parameter. If the first parameter is any value other than null, it returns unchanged value.

2- **NVL2(EXP1,EXP2,EXP3):**If the first column value is not null then it returns the value of the second column. If the first column value is null then it returns the value of third column.

3- **NULLIF(EXP1,EXP2):** It compares *expr1* and *expr2*. If *expr1* and *expr2* are equal, the NULLIF function returns NULL. Otherwise, it returns *expr1*.

Example: Run this command and study the results

```
select employee_id ,nvl (commission_pct,0), nvl2 (commission_pct, 10, 0), nullif(salary,9000)
from employees;
```

Exercise1: (From employees table).

1) Write a query to display last name, job id, commission for all employees who are not entitled to receive a commission.

2) Write a query to display last name, commission for all employees. If the employee does not receive a commission show 0 value instead of blank.

3) Write a query that returns salary if commission is null, otherwise it would return the result of summation of salary and commission. Name new column (income).

4) Write a query to display employee id, salary, commission_pct and a column called New Salary. New Salary is calculated by multiplying monthly salary with 12, plus multiplying commission_pct with salary. (USE appropriate null function to get correct results)

SQL character functions:

1-**Lower(string):** is used to convert all characters of a string to lower case.

2-**Upper(string)** :is used to convert all characters of a string to uppercase.

3-**INITCAP(string)** function sets the first character in each word to uppercase and the rest to lowercase.

4-**SUBSTR (string,Position,length**) is used to return a portion of string. ** When length is not specified, the entire string starting from the position-th character is returned.

5-**LENGTH(string) function**: returns the length of the specified string.

6-**CONCAT(string1,string2) function:** is used to concatenate two strings to form a single string.

7-**REPLACE(string_expression , string_pattern , string_replacement)**: In string_expression , find where string_pattern occurs, and replace it with string_replacement.

Example:

What is the output, after executing the following SQL commands?

1- SELECT LOWER (first_name) FROM employees WHERE First_Name = 'Alexander';

2- SELECT UPPER (first_name) FROM employees WHERE upper(First_Name) = 'Alexander';

3- SELECT SUBSTR (First_name, 3) FROM employees WHERE First_Name = 'Alexander';

4- SELECT SUBSTR (First_name, 3,4) FROM employees WHERE First_Name = 'Alexander';

5- SELECT length(CONCAT (first_name,'**')) FROM employees WHERE First_Name = 'Alexander';

6- SELECT REPLACE (first_name,'et','eee') FROM employees;

Exercise2: (From employees table).

1) Write a query to display employee_id, last name and department_id for employee who is last name is higgins.***Hint :- Do not use any character function .***

2) Write a query to display employee_id, last name and department_id for employee **higgins** .

Hint :- use lower function.

1) Write a query to display employee_id, last name and department_id for employee **HIGGINS**.***Hint :- use upper function.***

2) Write a query to display employee_id, last name and department_id for employee **higgins**.

Note :- use initcap function.

3) Write a query to display employee_id, first name and last name joined together (give concatenated string suitable name), job ID, length of employee last name for all employees who have REP, contained in the job ID starting at the fourth position of the job ID. ***Note :- use concat function, length function and substr function***

4) Write a query to retrieve last name, the first eight characters of last name and '_Us' joined together (the new column should be displayed in uppercase) for all employees who work in department 60.

5) Write a query to display first name, the last name, length of the first name, length of the last name, and the query should compare the length of first name and last name. When the lengths of names are equal then the null value is displayed. Otherwise the length of first name is displayed. **Hint :- use NULLif function in appropriate way.**

SQL Numeric functions:

We will study the following functions:

1) **Mod(N,M):** This function returns the remainder of N divided by M. Example:
SELECT MOD(29,3) FROM dual;

2) **ROUND(numeric value,[decimals])**: is used to round a numeric field to the number of decimals specified.
Example: What is the output?
SELECT ROUND (5.567, 2) FROM dual; SELECT ROUND(55.67, -1) FROM dual;

3) **TRUNC (numeric value, [decimals**]): returns a number truncated to a certain number of decimal places.
Example: What is the output?
SELECT TRUNC(125.815, 1) from dual; SELECT TRUNC(125.815, -1) from dual;

Exercise3: (from dual table).

1) Write a query to round (45.239) to two decimal places.
2) Write a query to round (1045.239) to nearest thousands.
3) Write a query to round (45.239) to zero decimal places.
4) Write a query to truncate (45.239) to two decimal places.
5) Write a query to truncate (45.239) to zero decimal places.
6) Write a query to truncate (45.239) to one decimal places to the left.

The Oracle TO_CHAR() function: converts a DATE or INTERVAL value to a string in a specified date format. The Oracle TO_CHAR() function is very useful for formatting the internal date data returned by a query in a specific date format.
Syntax:

TO_CHAR(<date>, '<format>')

Format:

MM	Numeric month (e.g., 07)
MON	Abbreviated month name (e.g., JUL)
MONTH	Full month name (e.g., JULY)
DD	Day of month (e.g., 24)
DY	Abbreviated name of day (e.g., FRI)
DAY	Full name of day (e.g., FRIDAY)
YYYY	4-digit year (e.g., 1998)
YY	Last 2 digits of the year (e.g., 98)
AM (or PM)	Meridian indicator
HH	Hour of day (1-12)
HH24	Hour of day (0-23)
MI	Minute (0-59)
SS	Second (0-59)

Example:

Select TO_CHAR(sysdate, 'yyyy/mm/dd') from dual;
Result: '2020/07/09'

Select TO_CHAR(sysdate, 'Month DD, YYYY') from dual; Result: 'July 09, 2003'

Select TO_CHAR(sysdate, 'MON DDth, YYYY') from dual; Result: 'JUL 09TH, 2003'

The Oracle TO_DATE() function: converts char of CHAR, VARCHAR2 datatype to a value of DATE datatype.
Syntax:

TO_Date(<date>, '<format>')

where the <format> string has the same options as in TO_CHAR function.

Exercise4: (use employees table)

1- Write a query to display hire date as 17 of JUN2020 12:00:00 AM.
2- Write a query to show the hire date in years only for all employees.
3- Write a query to display all employees who were hired before the year 2003.
Hint: solve using to_char function/ resolve it using like operator.

4- Write a SQL command to insert the following row into employees table.

Employee_id	First_name	Last_name	Email	Hire date	Department_id	JOB_ID
2020	Ali	Ali	**Ali@hu.edu.jo**	03.12/2012	50	ST_CLERK

5- Write a query to display the length of employee's first name, and the month(i.e. JULY) of hiring for all employees whose first name length is larger than 5.

SQL Aggregate functions: Aggregate functions return a single value.

- **SUM(column):** calculates the sum of values.
- **AVG (column)**: calculates the average of a set of values.
- **COUNT (column):** counts number of rows in a specified table.
- **MIN(column):** gets the minimum value in a set of values.
- **MAX(column):** gets the maximum value in a set of values.

Example:

What is the output, after executing the following SQL commands?

1- SELECT MIN(salary) FROM employees;

2- SELECT Max(salary) FROM employees;

3- SELECT count(*) FROM employees;

4- SELECT sum(salary) "total salaries of employees", AVG(salary) "Average salaries in the company" FROM employees;

Exercise 1: (From employees table).

1) Write a query to display the highest, lowest, sum and average salary for all employees. Label the columns Maximum, Minimum, Sum, and Average respectively .

2) Write a query to display the number of employees whose job id contains the string 'REP'.

3) Write a query to display the number of employees work in department 80.

4) Write a query to display the number of department values in the employees table without duplication.

5) Write a query to display number of employees who are not receiving a commission and were hired in 2004.

GROUP BY clause: is used in collaboration with the SELECT statement to arrange identical data into groups.The usage of SQL GROUP BY clause is, to divide the rows in a table into smaller groups. When some rows are retrieved from a grouped result against some condition, that is possible with HAVING clause. GROUP BY statement is often used with aggregate functions **Syntax**

```
SELECT <column_list>  FROM  < table name >[WHERE] <condition>
GROUP BY <columns>                          [HAVING] <condition>;
```

Example:

1)Write a query to display department_id ,highest salary, lowest salary , sum of salary and average of salary for each department.

Select department_id, max(salary),min(salary),sum(salary),avg(salary) from employees group by department_id;

2)Write a query to display department number, maximum salaries for those departments with a maximum salary greater than 10000$.

Select department_id, max(salary) from employees group by department_id having max(salary)>10000;

Exercise 2: (From employees table).

1)Write a query to display job id and number of employees who are entitled in each job.

2)Write a query to display job id and number of employees in each job. Show only the jobs that have more than 5 employees.

3)Write a query to display job id, total monthly salary for each job. Such that show all job id that contain REP string, total monthly salary for each job that has a total more than 1300. The output should be sorted descending by the total monthly salary.

A Subquery or Inner query or a Nested query: is a query within another SQL query and embedded within the WHERE clause. A subquery is used to return data that will be used in the main query as a condition to further restrict the data to be retrieved.

Example:

1) Write a query to display the employee ID, first name and job ID for all employees whose job ID is the same as that of employee 141.

```
Select employee_id, job_id from employees
Where job_id = (select job_id from employees where employee_id=141);
```

Exercise3(from employees table)

1) Write a query to display the name (first name and last name) for all employees who gets more salary than the employee id 163.

2) Write a query to display the employee last name, job ID and salary for all employees whose salary is equal to minimum salary.

3) Write a query to display department ID, minimum salary for each department that have minimum salary greater than that of department 50.

4) Write a query to display all information of last hired employee.

Hint: use appropriate aggregation function and use column employee_id.

5) Write a query to display the employee id, employee name (first name and last name) for all employees who earn more than the average salary

Congratulations on completing this database lab! Remember, this is just the beginning of your journey in learning and mastering databases. Keep exploring, practicing, and challenging yourself, because every step you take now builds the foundation for your future success in technology and development. Believe in your abilities and never stop learning.

Wishing you all the best in your studies and future endeavors.😊